

Individuell Labb 1k5: Robust AI-integration & Tillförlitlighet

Översikt över labben

I denna labb kommer du att bygga en Spring Boot-applikation på serversidan som fungerar som en säker proxy till en extern AI-tjänst (som OpenAI). Du ska inte bara integrera AI:n, utan också bygga ett defensivt "tillförlitlighetslager" (reliability layer) runt den för att hantera fel och begränsningar.

Förkunskapskrav:

- Java 21 & Spring Boot 3.x (Beroenden: Web, Validation)
- IDE (IntelliJ eller VS Code)
- En API-nyckel från en AI-leverantör (t.ex. OpenAI)

Steg 1: Säker konfiguration (Isolering av nycklar)

Koncept: Att hårdkoda API-nycklar är en kritisk säkerhetsrisk. Vi måste isolera dem från källkoden.

Vad du ska göra:

1. Öppna ditt Spring Boot-projekt och skapa en klass `AIClientService`.
2. Injicera din API-nyckel med Springs `@Value("${openai.api.key}")`-annotering så att den laddas från dina miljövariabler.
3. **Fail-Fast-implementering:** Lägg till en metod annoterad med `@PostConstruct` för att kontrollera om den injicerade nyckeln (`apiKey`) är null eller tom. Om den är det, kasta ett `IllegalStateException("CRITICAL: API key is missing.")`.

✓ **Förväntat resultat:** Om du försöker köra din Spring Boot-applikation utan att sätta miljövariabeln, bör applikationen krascha omedelbart vid uppstart med ditt anpassade felmeddelande.

Steg 2: Hantering av Timeouts

Koncept: LLM:er (Large Language Models) är beräkningstunga. Om AI-leverantören hänger sig kan det låsa dina servertrådar i över 30 sekunder och krascha din applikation.

Vad du ska göra:

1. Instansiera en `SimpleClientHttpRequestFactory` i din `AIClientService`.

2. Konfigurera en strikt `ConnectTimeout` (t.ex. 2000 ms) och `ReadTimeout` (t.ex. 8000 ms).
3. Bygg din Spring `RestClient` med denna anpassade request factory.

✓ **Förväntat resultat:** Din applikation har nu en strikt övre gräns för hur länge den kommer att vänta. Om AI:n inte svarar inom 8 sekunder avbryts anslutningen säkert.

Steg 3: Prompt Engineering & Deterministisk Utdata

Koncept: Du måste programmatiskt strukturera LLM-indatan för att få specifika, formaterade utdata (JSON) och förhindra "prompt injection".

Vad du ska göra:

1. Definiera en strikt systemprompt (System Prompt) som instruerar AI:n att extrahera data till ett specifikt JSON-schema (t.ex. för sentimentanalys eller schemaextraktion).
2. Instruera uttryckligen AI:n att *inte* inkludera markdown-formatering (som ````json`) eller konverserande text.
3. När du bygger JSON-payloaden för HTTP-anropet, sätt `"temperature"` till ett lågt värde (t.ex. `0.1`) för att prioritera deterministisk och repeterbar datagenerering.
4. Bifoga den dynamiska användarindatan via "User Role".

✓ **Förväntat resultat:** När anropet utförs ska AI:n konsekvent returnera en ren JSON-sträng som exakt matchar ditt schema.

Steg 4: Hantera Rate Limits (Exponential Backoff)

Koncept: Om du skickar för många anrop kommer AI-leverantören att returnera felet `HTTP 429 Too Many Requests`. Du måste hantera detta snyggt för att förhindra att trafiktoppar kraschar din app.

Vad du ska göra:

1. Omslut din `RestClient`-körning i en `for`-loop för att tillåta återförsök (t.ex. `int retries = 3`).
2. Implementera en **Exponential Backoff**-loop. Om svarsstatusen är `429`:
 - Logga en varning.
 - Använd `Thread.sleep(delay)` för att vänta (börja t.ex. med 1000 ms).
 - Multiplicera fördröjningen med 2 inför nästa iteration.
3. Om svaret är en framgång (`2xx`), returnera body:n och avbryt loopen (`break`).

✓ **Förväntat resultat:** Din kod kommer automatiskt att upptäcka rate limits (hastighetsbegränsningar), pausa och försöka igen utan att kasta ett ohanterat undantag till användaren.

Steg 5: Begränsa Hallucinationer & Validera Struktur

Koncept: Lita aldrig blint på råsträngsutdata från en AI. Syntaktiskt korrekta men faktamässigt felaktiga utdata kommer att förstöra underliggande logik.

Vad du ska göra:

1. Skapa en `ApiResponseDto` -klass som matchar ditt förväntade JSON-schema.
2. Lägg till Java Bean Validation-annoteringar (t.ex. `@NotNull`, `@Pattern`, `@Min`, `@Max`) för att framtvinga affärslogiska regler.
3. Använd Jacksons `ObjectMapper.readValue()` i din tjänst för att parse AI:ns strängsvar till din DTO.
4. Omslut parsningen i ett `try-catch` -block. Fånga upp `JsonProcessingException`.
5. Inuti catch-blocket, returnera ett säkert reservtillstånd (fallback state, t.ex. en standard-DTO) istället för att krascha.

✓ **Förväntat resultat:** Orimlig text fångas omedelbart upp av parsern. Om JSON-strukturen är giltig men datan bryter mot dina regler, fångas det av Bean Validation.

Steg 6: Testa Kantfall (Stresstestning)

Koncept: Du kan inte bevisa att dina felhanterare fungerar om du inte avsiktligt utlöser dem.

Vad du ska göra:

1. **Framtvinga Timeout:** Ändra tillfälligt din läs-timeout (read timeout) till `10` millisekunder och verifiera att ditt undantag (exception) utlöses.
2. **Framtvinga 429:** Skapa en tillfällig `@RestController` som alltid returnerar status `429`. Rikta din klient mot den för att se backoff-loopen köras i konsolen.
3. **Framtvinga Hallucination:** Ersätt det faktiska HTTP-svaret tillfälligt med en trasig sträng (t.ex. "Visst, här är din sammanfattning...") och verifiera att ditt `catch` -block fångar upp det.

Steg 7: Skriftlig Utvärdering av Tillförlitlighet (Krav för VG)

Koncept: En senior utvecklare förutser misslyckanden på ett ansvarsfullt sätt.

Vad du ska göra:

Skriv ett kort Markdown- eller PDF-dokument som täcker följande:

1. **Promptstrategi:** Dokumentera varför du valde dina specifika systeminstruktioner och hur du framtvingade JSON-formatet.
2. **Felhantering:** Beskriv din exakta strategi för timeouts, 429 rate limits och för att fånga upp validerings-/parsningsfel.

3. **Tillförlitlighetsbedömning:** Ge en ärlig kritik av LLM:en. Vilka är dess begränsningar i en produktionsmiljö, och hur tog din arkitektur höjd för dem?



Bedömningskriterier (Grading Principles)

Baserat på kursplanen "Mål 4 (AI-integration)" gäller följande kriterier för denna uppgift:

G Godkänd

AI-tjänsten är framgångsrikt integrerad som en applikationsfunktion. Applikationen konsumerar den externa LLM:en via ett HTTP-API, skickar kontext framgångsrikt och hämtar ett svar. API-nycklar hanteras säkert och är inte hårdkodade. Kodbasen kompileras och fungerar för standardscenarion ("happy path").

VG Väl Godkänd

För att uppnå VG för Mål 4 måste studenten visa **hög precision** i AI-integrationen. Detta kräver att:

- AI-tjänsten är integrerad på ett välmotiverat sätt med en **genomtänkt promptstrategi** (t.ex. för att effektivt framtvinga struktur och kontext).
- Applikationen har **robust felhantering av kantfall**, specifikt genom att implementera skydd mot rate limits (t.ex. Exponential Backoff), timeouts och AI-hallucinationer (säkerhetsnät via parsning/validering).
- Inlämningen inkluderar en **skriftlig utvärdering** som kritiskt bedömer AI-svarets tillförlitlighet och dess begränsningar i en produktionsmiljö.

Notera: För VG i kursen som helhet krävs att den studerande uppnår VG-nivå i denna uppgift (Individuell Labb 1k5).

Individuell Labb 1k5: Robust AI-Integration & Reliability

Lab Overview

In this lab, you will build a server-side Spring Boot application that acts as a secure proxy to an external AI service (like OpenAI). You will not only integrate the AI, but you will also build a defensive "reliability layer" around it.

Prerequisites:

- Java 21 & Spring Boot 3.x (Web, Validation dependencies)
- IDE (IntelliJ or VS Code)
- An API key from an AI provider (e.g., OpenAI)

Step 1: Secure Configuration (Isolating Credentials)

Concept: Hardcoding API keys is a critical security vulnerability. We must isolate them from the source code.

What to do:

1. Open your Spring Boot project and create an `AIClientService` class.
2. Inject your API key using Spring's `@Value("${openai.api.key}")` annotation so it loads from your environment variables.
3. **Fail-Fast Implementation:** Add a method annotated with `@PostConstruct` to check if the injected `apiKey` is null or blank. If it is, throw an `IllegalStateException("CRITICAL: API key is missing.")`.

✓ **Expected Output:** If you try to run your Spring Boot app without setting the environment variable, the application should crash immediately on startup with your custom error message.

Step 2: Managing Timeouts

Concept: LLMs are computationally heavy. If the AI provider hangs, it can tie up your server threads for 30+ seconds and crash your application.

What to do:

1. In your `AIClientService` , instantiate a `SimpleClientHttpRequestFactory` .
2. Configure a strict `ConnectTimeout` (e.g., 2000ms) and `ReadTimeout` (e.g., 8000ms).
3. Build your Spring `RestClient` using this custom request factory.

✅ **Expected Output:** Your application now has a strict upper limit on how long it will wait. If the AI doesn't respond within 8 seconds, the connection is safely aborted.

Step 3: Prompt Engineering & Deterministic Output

Concept: You must programmatically structure the LLM input to yield specific, formatted outputs (JSON) and prevent prompt injection.

What to do:

1. Define a strict System Prompt that instructs the AI to extract data into a specific JSON schema (e.g., for sentiment analysis or schedule extraction).
2. Explicitly instruct the AI *not* to include markdown formatting (like ````json`) or conversational text.
3. When building the JSON payload for the HTTP API request, set the `"temperature"` to a low value (e.g., `0.1`) to prioritize deterministic, repeatable data generation.
4. Append the dynamic user input via the User Role.

✅ **Expected Output:** When executing the request, the AI should consistently return a clean JSON string matching your exact schema.

Step 4: Handling Rate Limits (Exponential Backoff)

Concept: If you send too many requests, the AI provider will return an `HTTP 429 Too Many Requests` error. You must handle this gracefully to prevent traffic spikes from crashing your app.

What to do:

1. Wrap your `RestClient` execution inside a `for` loop to allow for retries (e.g., `int retries = 3`).
2. Implement an **Exponential Backoff** loop. If the response status is `429` :
 - Log a warning.
 - Use `Thread.sleep(delay)` to wait (e.g., start with 1000ms).
 - Multiply the delay by 2 for the next iteration.
3. If the response is a `2xx` success, return the body and break the loop.

✓ **Expected Output:** Your code will automatically detect rate limits, pause, and retry without throwing an unhandled exception to the user.

Step 5: Mitigating Hallucinations & Validating Shape

Concept: Never blindly trust raw string output from an AI. Syntactically correct but factually incorrect outputs will break downstream logic.

What to do:

1. Create an `AiResponseDto` class matching your expected JSON schema.
2. Add Java Bean Validation annotations (e.g., `@NotNull`, `@Pattern`, `@Min`, `@Max`) to enforce business logic rules.
3. In your service, use Jackson's `ObjectMapper.readValue()` to parse the AI's string response into your DTO.
4. Wrap the parsing in a `try-catch` block. Catch `JsonProcessingException`.
5. Within the catch block, return a safe fallback state (e.g., a default DTO) instead of crashing.

✓ **Expected Output:** Nonsensical text is immediately caught by the parser. If the JSON is valid but the data violates your rules, the Bean Validation catches it.

Step 6: Testing the Edge Cases (Stress Testing)

Concept: You cannot prove error handlers work unless you deliberately trigger them.

What to do:

1. **Force Timeout:** Temporarily change your read timeout to `10` milliseconds and verify your exception triggers.
2. **Force 429:** Create a temporary `@RestController` that always returns `429`. Point your client to it to watch the backoff loop execute.
3. **Force Hallucination:** Replace the actual HTTP response with a broken string (e.g., "Sure, here is your summary...") and verify the `catch` block intercepts it.

Step 7: Written Reliability Assessment

Concept: A senior developer anticipates failure responsibly.

What to do:

Write a short Markdown or PDF document covering the following:

1. **Prompt Strategy:** Document why you chose your specific system instructions and how you enforced the JSON format.

2. **Error Mitigation:** Detail your exact approach for timeouts, 429 rate limits, and catching validation/parsing failures.
3. **Reliability Assessment:** Provide an honest critique of the LLM. What are its limitations in a production environment, and how did your architecture anticipate them?



Grading Principles (Bedömningskriterier)

Based on the course syllabus "Mål 4 (AI-integration)", the following criteria apply for this assignment:

G Godkänd (Pass)

The AI service is successfully integrated as an application feature. The application consumes the external LLM via an HTTP API, successfully passes context, and retrieves a response. API keys are managed securely and are not hardcoded. The codebase compiles and functions for standard "happy path" scenarios.

VG Väl Godkänd (Pass with Distinction)

To achieve VG for Mål 4, the student must demonstrate **high precision** in the AI integration. This requires that:

- The AI service is integrated in a well-motivated manner with a **carefully considered prompt strategy** (e.g., enforcing structure and context effectively).
- The application features **robust error handling for edge cases**, specifically implementing defenses against rate limits (e.g., Exponential Backoff), timeouts, and AI hallucinations (parsing/validation safety nets).
- The submission includes a **written evaluation** critically assessing the reliability of the AI's responses and its limitations in a production context.

Note: Achieving VG on Individuell Labb 1k5 is required to be eligible for a final VG grade in the overall course.